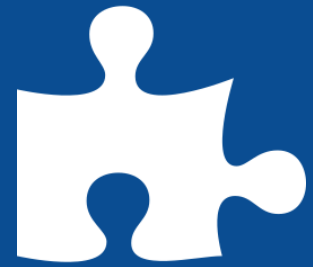
INSTITUTO NACIONAL DE APRENDIZAJE

Núcleo Comercio y Servicios · Subsector Informática y Comunicación

Diseño de algoritmos

Sesión 21 · Funciones: invocación, retorno, alcance y bibliotecas

UA2 · Programación estructurada | Módulo CSTI12010



Persona facilitadora

Giovanni Antonio Coto Calderón

SABER 1

¿Qué es una función?



Una **función** es un bloque de instrucciones con **nombre** que **recibe datos** (parámetros), hace una tarea y puede **devolver un resultado** (return). Se escribe **una vez** y se **llama** cuantas veces haga falta.



Dividir

Un problema grande en partes pequeñas.



Reutilizar

Escribir una vez, usar muchas.



Probar

Cada parte por separado.



Leer

main se lee como un índice.

Prototipo, llamada y definición

El **prototipo** anuncia la función; la **llamada** le envía datos y recibe el resultado; la **definición** tiene el código. **return** devuelve el valor.

iva.c

```c
#include <stdio.h>
#define TASA_IVA 0.13
double calcularIVA(double monto); //Prototipo. Nombre de la función + parámetro, compuesto de la variable y su tipo entre paréntesis
int main(void) {
    double precio = 15000;
    double iva = calcularIVA(precio);  //Llamada: toma el valor de precio (15000). Ese valor se copia en el parámetro monto y el programa salta a la línea 11.
    printf("IVA de %.2f: %.2f\n", precio, iva);
    printf("IVA de 2500: %.2f\n", calcularIVA(2500));
    return 0;
}
double calcularIVA(double monto) {//Definicion: mismo encabezado del prototipo, pero sin ; y con llaves, porque aquí está el código. Monto existe solo dentro de esta función.

    return monto * TASA_IVA; //La función calcula 15000 × 0.13 = 1950 y lo devuelve con return.
}
```

Salida real (gcc)

```
IVA de 15000.00: 1950.00
IVA de 2500: 325.00
```

Las partes de una función

Parte	Ejemplo	Para qué sirve
Tipo de retorno	`double calcularIVA(…)`	Qué tipo de dato devuelve
Nombre	`calcularIVA`	Verbo + qué hace
Parámetros	`(double monto)`	Datos que recibe (copias)
return	`return monto * 0.13;`	Devuelve el resultado y termina
void	`void mostrarMenu(void)`	No devuelve / no recibe nada
Prototipo	`double calcularIVA(double monto);`	Anuncia la función antes de usarla
Argumento	`calcularIVA(15000)`	Valor que se envía en la llamada

Funciones que no devuelven nada

Con **void** la función **solo hace** algo (mostrar, dibujar). Una función puede **llamar a otra**: mostrarTitulo usa mostrarLinea.

titulo.c (sin #include)

```c
void mostrarLinea(int largo) {
    int i;
    for (i = 1; i <= largo; i++) {
        printf("-");
    }
    printf("\n");
}
void mostrarTitulo(void) {
    mostrarLinea(19);
    printf("COMPROBANTE DE PAGO\n");
    mostrarLinea(19);
}
int main(void) {
    mostrarTitulo();
    return 0;
}
```

Salida real (gcc)

```
-------------------
COMPROBANTE DE PAGO
-------------------
```

Una validación, muchos usos

La validación con **do … while** de la sesión 19 se escribe **una vez** en **leerEntero** y se usa con **límites distintos**.

leer.c

```c
#include <stdio.h>
int leerEntero(int minimo, int maximo) {
    int valor;
    do {
        printf("Valor (%d a %d): ", minimo, maximo);
        scanf("%d", &valor);
    } while (valor < minimo || valor > maximo);
    return valor;
}
int main(void) {
    int edad = leerEntero(15, 99);
    int opcion = leerEntero(0, 3);
    printf("Edad: %d  Opcion: %d\n", edad, opcion);
    return 0;
}
```

Salida real (120, 20, 5, 2)

```
Valor (15 a 99): 120
Valor (15 a 99): 20
Valor (0 a 3): 5
Valor (0 a 3): 2
Edad: 20  Opcion: 2
```

Variables locales y globales

Una variable **local** existe solo dentro de su función. Una **global** (fuera de todas) la ven todas. Dos funciones pueden tener locales con **el mismo nombre** sin mezclarse.

alcance.c

```c
#include <stdio.h>
int contador = 0;               // GLOBAL: la ven todas
void registrar(void) {
    int temporal = 10;          // LOCAL de registrar
    contador++;
    printf("registrar: contador=%d temporal=%d\n",
           contador, temporal);
}
int main(void) {
    int temporal = 99;          // LOCAL de main (otra)
    registrar();
    registrar();
    printf("main: contador=%d temporal=%d\n",
           contador, temporal);
    return 0;
}
```

Salida real (gcc)

```
registrar: contador=1 temporal=10
registrar: contador=2 temporal=10
main: contador=2 temporal=99
```

SABER 3

Variables automáticas y static

Una local **automática** se crea en cada llamada y desaparece al salir. Con **static** se crea **una sola vez** y **conserva su valor** entre llamadas.

static.c (fragmento)

```c
void automatica(void) {
    int veces = 0;              // se crea en cada llamada
    veces++;
    printf("automatica: %d\n", veces);
}
void estatica(void) {
    static int veces = 0;       // conserva su valor
    veces++;
    printf("estatica:   %d\n", veces);
}
int main(void) {
    int i;
    for (i = 1; i <= 3; i++) {
        automatica();
        estatica();
    }
```

Salida real (gcc)

```
automatica: 1
estatica:   1
automatica: 1
estatica:   2
automatica: 1
estatica:   3
```

Funciones numéricas: math.h

Función	Qué hace	Ejemplo	Resultado
`sqrt(x)`	Raíz cuadrada	`sqrt(144)`	`12`
`pow(b, e)`	Potencia (C no tiene ^)	`pow(2, 10)`	`1024`
`fabs(x)`	Valor absoluto de un real	`fabs(-7.5)`	`7.5`
`round(x)`	Redondea al entero más cercano	`round(2.5)`	`3`
`ceil(x)`	Redondea hacia arriba	`ceil(4.1)`	`5`
`floor(x)`	Redondea hacia abajo	`floor(4.9)`	`4`

Reciben y devuelven **double**. En Linux se compila con **-lm**; con MSYS2 en Windows no hace falta.

math.h en acción

matematicas.c

```c
#include <stdio.h>
#include <math.h>
int main(void) {
    printf("sqrt(144)  = %.2f\n", sqrt(144));
    printf("pow(2, 10) = %.0f\n", pow(2, 10));
    printf("fabs(-7.5) = %.1f\n", fabs(-7.5));
    printf("round(2.5) = %.0f\n", round(2.5));
    printf("ceil(4.1)  = %.0f\n", ceil(4.1));
    printf("floor(4.9) = %.0f\n", floor(4.9));
    return 0;
}
```

Salida real (gcc)

```
sqrt(144)  = 12.00
pow(2, 10) = 1024
fabs(-7.5) = 7.5
round(2.5) = 3
ceil(4.1)  = 5
floor(4.9) = 4
```

Funciones de caracteres: ctype.h

isalpha, **isdigit** e **isspace** preguntan qué tipo de carácter es (1 o 0). **toupper** y **tolower** lo convierten.

caracteres.c

```c
#include <stdio.h>
#include <ctype.h>
int main(void) {
    char texto[] = "Ced 1-0234";
    int i, letras = 0, digitos = 0;
    for (i = 0; texto[i] != '\0'; i++) {
        if (isalpha(texto[i])) { letras++; }
        if (isdigit(texto[i])) { digitos++; }
        putchar(toupper(texto[i]));
    }
    printf("\nLetras: %d  Digitos: %d\n", letras, digitos);
    return 0;
}
```

Salida real (gcc)

```
CED 1-0234
Letras: 3  Digitos: 5
```

stdlib.h: rand, abs y atoi

rand() da un número pseudoaleatorio; **rand() % 6 + 1** simula un dado (**LanzarDado**, sesión 2). **srand** fija la semilla; **abs** y **atoi** completan la biblioteca.

dado.c

```c
#include <stdio.h>
#include <stdlib.h>              // rand, srand, abs, atoi
int main(void) {
    int i;
    srand(2026);                 // semilla fija
    for (i = 1; i <= 5; i++) {
        printf("%d ", rand() % 6 + 1);  // dado 1 a 6
    }
    printf("\nabs(-8) = %d\n", abs(-8));
    printf("atoi(\"2026\") + 1 = %d\n", atoi("2026") + 1);
    return 0;
}
```

Salida real (Linux)

```
2 1 2 4 5
abs(-8) = 8
atoi("2026") + 1 = 2027
```